

OS Exercise 3 — Part A: Theory

Course: Operating Systems — Reichman University

Due date: 27.06.2026 — submit via INGINious

Names: Ariel Arusy, Nathaniel Kinzelberg

IDs: 323978494, 809136

TQ1 — Scheduling: A Multi-Level Feedback Queue (MLFQ) Variant

Consider the following **variant** of a Multi-Level Feedback Queue (MLFQ) scheduler with three priority queues:

Queue	Quantum
Q0	10 ms
Q1	20 ms
Q2	40 ms

Rules:

- New threads start at Q0 (highest priority).
- A thread that uses its full quantum is demoted one level (a thread already in Q2 stays in Q2).
- A thread that yields or blocks before its quantum expires stays at its current level.
- The scheduler always dispatches from the highest-priority non-empty queue. Within a queue, threads are served in FIFO order.

Note: This is one specific variant of MLFQ; other variants behave differently (for example, *promoting* a thread that yields). Answer according to the rules stated above.

Two threads arrive at $t = 0$, each needing 1000 ms of CPU time in total:

- **Thread A** is genuinely CPU-bound: it runs until it is preempted.
- **Thread B** is also CPU-bound, but tries to *masquerade* as an I/O-bound thread: it calls `yield()` after exactly 9.9 ms of every quantum it receives.

Task 1 — Steady-state queue placement

Once the system reaches a steady state (after the initial scheduling rounds), in which queue is each thread?

Answer:

In steady state **Thread B stays in Q0** (the highest-priority queue) and **Thread A sits in Q1**.

- B yields after 9.9 ms, which is *less than the 10 ms Q0 quantum*, so it never consumes a full quantum and is therefore never demoted — it stays in Q0.
- A is CPU-bound: it uses its full Q0 quantum once and is demoted to Q1. From that point on Q0 is never empty (B keeps returning to it), so the scheduler — which always serves the highest-priority non-empty queue — never dispatches A again. A is therefore parked in Q1 and, never running, is never demoted further.

Task 2 — CPU share

At most how many full quanta does Thread A complete before Thread B takes over the CPU? In steady state, while both threads are still alive, what percentage of CPU time does Thread A receive? Explain in one sentence.

Answer:

Thread A completes **at most 1 full quantum** (its single initial 10 ms quantum in Q0) before Thread B takes over the CPU. In steady state, while both threads are alive, **Thread A receives ≈ 0 %** of the CPU — it starves — because Thread B never leaves Q0 and the scheduler always dispatches from the highest-priority non-empty queue (Q0), so A waiting in Q1 is never scheduled.

Task 3 — Incorrect assumption

In one sentence: what does this MLFQ variant incorrectly assume about Thread B?

Answer:

It incorrectly assumes that because Thread B keeps yielding before its quantum expires it must be I/O-bound / interactive and so deserves to keep high priority — when in fact B is CPU-bound and is merely gaming the "yield => stay high" heuristic.

Task 4 — Demotion

Under these rules, can Thread B ever be demoted? Why or why not? (One sentence.)

Answer:

No — demotion happens only when a thread uses up a full quantum, and B always yields after 9.9 ms, which is shorter than the quantum of every queue (10 / 20 / 40 ms), so it never triggers a demotion.

TQ2 — Memory: Demand Paging and Page Replacement

A process runs on a machine with a page size of 4 KB and **demand paging**: a virtual page is brought into a physical frame only when it is first accessed. Only **3 physical frames** are available to this process.

Each page table entry (PTE) has a **valid bit** (is the page in a frame?), a **reference bit** (used by Clock), and a **dirty bit** (was the page modified since it was loaded?).

The process accesses this **reference string**, from left to right:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Clock (approximate-LRU) convention — use exactly this:

- The 3 frames form a circle. The first three faults fill Frame 0, Frame 1, Frame 2 in order.
- A page loaded into a frame gets reference bit = **1**.
- On a **hit**, set that page's reference bit to **1**.
- On a **fault** when all frames are full, start at the hand. If the page it points to has reference bit **1**, clear it to **0** and advance the hand, repeating until a page with reference bit **0** is found. Evict that page, load the new page in its place (reference bit **1**), and advance the hand one position past it.

In the Clock table, the **Hand** column gives the hand position *after* the access has been processed.

Task 1 — Page fault condition

Fill in the blanks: a page fault occurs when the accessed page's PTE has valid bit = ____ . The CPU then traps to the OS, which loads the page into a ____ .

Answer:

A page fault occurs when the accessed page's PTE has valid bit = **0**. The CPU then traps to the OS, which loads the page into a free **physical frame** (page frame in RAM).

Task 2 — LRU simulation

Simulate the reference string under **LRU** by completing the table, then give the total number of page faults.

Access	1	2	3	4	1	2	5	1	2	3	4	5
Fault? (F/H)	F											
Resident pages	{1}											

Total LRU page faults: ____

Answer:

LRU evicts the least-recently-used resident page on each fault.

Access	1	2	3	4	1	2	5	1	2	3	4	5
Fault? (F/H)	F	F	F	F	F	F	F	H	H	F	F	F
Resident pages	{1}	{1,2}	{1,2,3}	{2,3,4}	{3,4,1}	{4,1,2}	{1,2,5}	{1,2,5}	{1,2,5}	{1,2,3}	{2,3,4}	{3,4,5}

Evictions: 4->evict 1, 1->evict 2, 2->evict 3, 5->evict 4, 3->evict 5, 4->evict 1, 5->evict 2.

Total LRU page faults: 10 (only accesses 8 and 9 — the second 1 and 2 — are hits).

Task 3 — Clock simulation

Simulate the same reference string under **Clock** (convention above) by completing the table, then give the total number of page faults. Each frame cell is (page, reference bit).

Access	F/H	Frame 0	Frame 1	Frame 2	Hand
1	F	(1,1)	—	—	1
2					2
3					0
4					
1					
2					
5					
1					
2					
3					
4					
5					

Total Clock page faults: ____

Answer:

Each cell is (page, reference bit). The **Hand** column is the hand position *after* the access. On a fault with all frames full, the hand sweeps clearing reference bits until it finds one already 0, evicts there, loads the new page (ref = 1), and advances one position past it.

Access	F/H	Frame 0	Frame 1	Frame 2	Hand
1	F	(1,1)	—	—	1
2	F	(1,1)	(2,1)	—	2
3	F	(1,1)	(2,1)	(3,1)	0
4	F	(4,1)	(2,0)	(3,0)	1
1	F	(4,1)	(1,1)	(3,0)	2
2	F	(4,1)	(1,1)	(2,1)	0
5	F	(5,1)	(1,0)	(2,0)	1
1	H	(5,1)	(1,1)	(2,0)	1
2	H	(5,1)	(1,1)	(2,1)	1
3	F	(5,0)	(3,1)	(2,0)	2
4	F	(5,0)	(3,1)	(4,1)	0
5	H	(5,1)	(3,1)	(4,1)	0

Walk-through of the non-trivial faults:

- **Access 4:** hand at 0; sweep clears bits of 1, 2, 3 (->0) returning to frame 0 whose bit is now 0; evict 1, load 4, hand -> 1.
- **Access 1:** hand at 1; frame 1 holds (2,0) => evict 2 immediately, load 1, hand -> 2.
- **Access 2:** hand at 2; frame 2 holds (3,0) => evict 3, load 2, hand -> 0.
- **Access 5:** hand at 0; sweep clears 4, 1, 2 returning to frame 0 (now 0); evict 4, load 5, hand -> 1.
- **Access 3:** hand at 1; sweep clears 1, 2, 5 returning to frame 1 (now 0); evict 1, load 3, hand -> 2.
- **Access 4:** hand at 2; frame 2 holds (2,0) => evict 2, load 4, hand -> 0.
- **Access 5 (last):** page 5 is still resident in frame 0 => **hit** (set its bit to 1).

Total Clock page faults: 9 (hits at accesses 8, 9, and 12).

Task 4 — LRU vs Clock comparison

Exact LRU produced __ faults, Clock produced __ faults. **True or False:** an approximate policy like Clock can never cause fewer faults than exact LRU. Justify in one sentence.

Answer:

Exact LRU produced **10** faults, Clock produced **9** faults. The claim "an approximate policy like Clock can never cause fewer faults than exact LRU" is **False** — this very reference string is a counterexample (Clock 9 < LRU 10): only the optimal (OPT/Bélády) policy is guaranteed minimal, so neither LRU nor its Clock approximation is optimal, and on particular access patterns Clock can happen to evict more wisely and beat LRU.

Task 5 — Dirty vs clean eviction

When the OS evicts a victim, exactly one case requires an extra write to swap before the frame is reusable: a **clean** page or a **dirty** page? (Pick one.) In one sentence, why?

Answer:

A **dirty** page. Because it was modified since it was loaded, the copy in the frame differs from the copy in swap/backing store, so the OS must write it back to swap before the frame can be reused. A clean page is identical to its backing-store copy, so it can simply be discarded with no write.

TQ3 — File Systems: Links, Partitions, and Indirect Blocks

Alice's Linux laptop has two separate **ext4** partitions: one mounted at `/home`, the other at `/data`. She has a 5 GiB file at `/home/alice/report.pdf`, plus a **hard link** to the same file at `/home/alice/backup/report.pdf`.

The two commands below are independent alternatives. Each starts from the original state above.

- **Command A:** `mv /home/alice/report.pdf /home/alice/archive/report.pdf`
- **Command B:** `mv /home/alice/report.pdf /data/report.pdf`

Task 1 — What mv copies

Command A copies ___ bytes of the file's content; Command B copies ___ bytes. In one sentence, what does Command A actually change on disk?

Answer:

Command A copies **0** bytes; Command B copies **5 GiB = $5 \times 2^{30} = 5,368,709,120$** bytes. Command A is a rename *within the same ext4 partition*, so on disk it only rewrites directory entries — removing the `report.pdf` entry in `alice/` and adding one in `alice/archive/`, both still pointing at the same inode — while the inode and its data blocks are left completely untouched.

Task 2 — Hard link count after Command B

Originally, two hard links point to the file's inode. After **Command B** removes the entry `/home/alice/report.pdf`, how many hard links still point to that `/home` inode? ___ Running `cat /home/alice/backup/report.pdf` then shows (pick one): original content / empty output / error. In one sentence, why?

Answer:

1 hard link still points to the `/home` inode (the `backup/report.pdf` entry). `cat /home/alice/backup/report.pdf` shows the **original content** — because Command B copied the bytes to a *new* inode on `/data` and only unlinked the `report.pdf` name on `/home`, the original inode's link count drops from 2 to 1 (not 0), so its data blocks are not freed and `backup/report.pdf` still references them.

Task 3 — Hard links across partitions

True or False: a hard link can point to a file whose inode lives on a different partition. Then, in one sentence, explain why `ln /home/alice/backup/report.pdf /data/hardlink.pdf` fails.

Answer:

False. `ln /home/alice/backup/report.pdf /data/hardlink.pdf` fails because a hard link is merely a directory entry holding an inode number, and inode numbers are only meaningful within their own filesystem; `/home` and `/data` are separate partitions with independent inode tables, so a directory entry on `/data` cannot reference an inode living on `/home` (the kernel rejects it with `EXDEV`, "Invalid cross-device link").

Task 4 — Symbolic link after Command B

Suppose `/home/alice/backup/report.pdf` were a **symbolic link** to `/home/alice/report.pdf` instead of a hard link. After **Command B**, `cat /home/alice/backup/report.pdf` shows (pick one): original content / empty output / error (broken link). In one sentence, why?

Answer:

Error (broken link). A symbolic link stores a *pathname* (`/home/alice/report.pdf`), not a reference to an inode; after Command B moves the original file off `/home`, that pathname no longer resolves, so dereferencing the dangling symlink fails with "No such file or directory."

Task 5 — Indirect block arithmetic

The inodes on /data use the structure taught in class: **12 direct, 1 single-indirect, 1 double-indirect, 1 triple-indirect** block pointers, with **4 KiB** blocks and **4-byte** pointers. A file occupies **5 GiB**. ("Data blocks" = blocks holding file content, not the indirect pointer blocks.)

1. How many block pointers fit in one indirect block? __
2. How many data blocks does the 5 GiB file occupy? __
3. How many data blocks can be reached using only the direct + single-indirect + double-indirect pointers? __
4. Is the triple-indirect pointer needed for this file? (yes/no)

Answer:

Block = 4 KiB = 4096 B, pointer = 4 B.

1. **Pointers per indirect block:** $4096 / 4 = 1024$.
2. **Data blocks in the 5 GiB file:** $5 \times 2^{30} / 2^{12} = 5 \times 2^{18} = 1,310,720$ data blocks.
3. **Reachable via direct + single-indirect + double-indirect:** $12 + 1024 + 1024^2 = 12 + 1024 + 1,048,576 = 1,049,612$ data blocks.
4. **Yes** — the file needs 1,310,720 blocks but only 1,049,612 are reachable without the triple-indirect pointer ($1,310,720 > 1,049,612$), so the **triple-indirect pointer is required**.